

GOETHE-UNIVERSITÄT FRANKFURT
AM MAIN

FACHBEREICH INFORMATIK

Instruction Robustness in
RAG/GraphRAG Agent
Pipelines

Bachelor Thesis

Name: Christian Block

Matriculation Number: 6522508

First examiner: Prof. Dr. Visvanathan Ramesh

Date of Submission:

Erklärung zur Abschlussarbeit

gemäß § 35, Abs. 16 der Ordnung für den Bachelorstudiengang Informatik vom 17. Juni 2019:

Hiermit erkläre ich (Nachname, Vorname) **Block, Christian**, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel verfasst habe. Ich bestätige außerdem, dass die vorliegende Arbeit nicht, auch nicht auszugsweise, für eine andere Prüfung oder Studienleistung verwendet wurde. Zudem versichere ich, dass alle eingereichten schriftlichen gebundenen Versionen meiner vorliegenden Bachelorarbeit mit der digital eingereichten elektronischen Version meiner Bachelorarbeit übereinstimmen.

Frankfurt am Main, den DATUM _____

Unterschrift der/des Studierenden

Abstract

Retrieval-augmented generation (RAG) combines information retrieval with language generation, so that a language model can condition its output on external and updatable evidence. The system prompt specifies the instructions, constraints, and output format the generator should adhere to. Because it is written in natural language, its effect on system behaviour is hard to predict and hard to verify. Prior work on prompt sensitivity in language models has studied that effect outside retrieval-augmented systems. This thesis addresses the resulting gap by asking to what extent systematic changes in the system prompt influence the behaviour and outcomes of a RAG system.

The thesis builds a formal model of a RAG system based on the formulation of Lewis et al. (2020). A retriever assigns probabilities $p_\eta(z \mid q, D)$ to documents from the knowledge base, and a generator produces the answer token by token. The system prompt S enters as an explicit variable, $p_\theta(y_i \mid q, C, y_{1:i-1}, S)$. It is modelled as a set of checkable requirements Σ written in a wording φ , such that $S = \rho(\Sigma, \varphi)$. Each of three published baseline prompts is written in four wordings: its canonical form and three rewordings, each of which changes either the vocabulary, the sentence structure or the layout. Every rewording leaves the requirements fixed and changes only φ .

The properties a good answer must have are fixed beforehand: it must be grounded in the retrieved evidence, relevant to the question asked, correct with respect to the reference answer, and compliant with the instructions it was given. The knowledge base and the query

set come from an established question-answer dataset, so that the measurement is reproducible. Only the system prompt is manipulated. The language model, the embedding model, the vector index, the chunking strategy and the retrieval depth are held fixed. The generator therefore receives identical passages in every condition. Any difference in the output can be attributed to the generator alone. Generations are repeated to account for decoding randomness.

The null hypothesis of equal behaviour across wordings is rejected. The clearest effects are on instruction adherence, which three rewordings raised by up to 26 percentage points and one lowered by up to 12. Correctness, measured by ROUGE-L, BLEU-1 and BERTScore, changed by a few percentage points, mostly because the answers changed in length. Faithfulness and answer relevance showed no consistent change.

Acknowledgments

I want to express my sincere gratitude to Professor Ramesh for his guidance and mentorship throughout this thesis and Professor Gao for being the second supervisor of this thesis.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Research Objective	1
1.3	Thesis Structure	2
	Notation	3
2	Preliminaries	6
2.1	Path towards RAG	6
2.2	Components of RAG	7
2.2.1	Knowledge Base	7
2.2.2	Retriever	8
2.2.3	Generator	9
2.2.4	System Prompt	10
2.3	Advanced RAG Architectures	10
3	Literature Review	12
3.1	Prompt Sensitivity and Instruction-Following in Lan- guage Models	12
3.2	Retrieval-Augmented Generation: Architecture, Evalu- ation, and Failure Modes	14
3.3	Synthesis and Research Gap	15
4	Formal Model of a RAG System	17
4.1	Modelling Objective	17
4.2	Probabilistic Retrieval	18
4.3	Prompt-Conditioned Generation	18

4.4	From One Document to the Whole Context	19
4.4.1	RAG-Sequence	19
4.4.2	RAG-Token	20
4.5	The System Prompt: Requirements and Wording . . .	21
4.6	The System Prompt as an Intervention	23
4.7	The Causal Effect	24
4.8	Grounding the Knowledge Base in an Established Dataset	25
4.9	What Counts as a Good Answer	26
5	Experimental Setup	28
5.1	Research Question and Hypothesis	28
5.2	Overview of the Experiment	29
5.3	System-Prompt Conditions	30
5.4	Variables and Controlled Factors	35
5.5	Knowledge Base and Query Set	37
5.6	Experimental Procedure	38
5.6.1	Stage 1: Retrieval and the Retrieval Gate . . .	38
5.6.2	Stage 2: Generation	39
5.7	Evaluation	39
5.7.1	Six Measures for Four Properties	40
5.7.2	The Judge	41
5.7.3	Estimating the Effect of a Rewording	42
6	Results and Discussion	43
6.1	Descriptive Results	43
6.2	The Effect of Rewording	45
6.2.1	Instruction Adherence	45
6.2.2	Faithfulness	46
6.2.3	Answer Relevance	46
6.2.4	Correctness	46

6.3	The Verdict on the Hypothesis	49
6.4	Limitations	49
7	Conclusion and Future Work	51
7.1	Conclusion	51
7.2	Future Work	52

1 Introduction

1.1 Background and Motivation

Retrieval-Augmented Generation (RAG) systems combine information retrieval with language generation. A RAG system takes the user query, retrieves relevant information from an external knowledge base and gives that information to a language model with instructions for generating the final response.

The system prompt is where the intended behaviour of the system is specified, and it shapes the output the system produces. Whereas the retrieved evidence varies with the query and the knowledge base, the system prompt specifies how the model should behave, which instructions and constraints apply, and which output format it should produce. Because it is written in natural language, its effect on system behaviour is difficult to predict. Its influence therefore has to be examined through controlled experiments.

1.2 Research Objective

The central research question is the following:

To what extent do systematic changes in the system prompt influence the behaviour and outcomes of a RAG system when the user input, knowledge base, retrieval mechanism, and underlying language model

are held fixed?

Answering it requires a formal model of the RAG system that identifies the system prompt as a controlled parameter, and an experimental design that holds every other source of variation fixed.

This thesis develops both for the simplest retrieval-augmented architecture. In that architecture the system prompt reaches the answer along one path, through the generator. A measured effect can therefore be read as an effect on generation, and not as a change in what was retrieved.

1.3 Thesis Structure

This thesis has seven chapters. Chapter 2 outlines why retrieval-augmented generation was developed and introduces the components of a RAG system. Chapter 3 reviews the literature and identifies the research gap. Chapter 4 builds the formal model used here. It takes the probabilistic formulation of Lewis et al. (2020) and extends it so that the system prompt becomes an explicit variable of the generator. It also instantiates the knowledge base with an established dataset and defines what counts as a good answer. Chapter 5 describes the experimental setup. Chapter 6 presents and discusses the results and states the limits of the design. Chapter 7 draws conclusions and outlines directions for future work.

Notation

The notation follows Lewis et al. (2020) for the RAG system and the intervention calculus of Pearl (2009) for the causal statements.

The system and its components

- $q \in \mathcal{Q}$ denotes a user query, and $P_{\mathcal{Q}}$ the distribution over the query space $\mathcal{Q}$ from which queries are drawn.
- $D = \{d_1, d_2, \dots, d_N\}$ denotes the knowledge base, where each d_n is a document and N is the number of documents.
- S denotes a system prompt.
- C denotes the context supplied to the language model: the passages it reads, in the order it reads them.
- $y = (y_1, \dots, y_L)$ denotes the generated system output, a sequence of L tokens; $y_{1:i-1}$ denotes the tokens produced before position i .
- R and G denote the retrieval and generation functions of the pipeline, and χ the chunking strategy by which documents are divided into retrievable units.
- $v \in \mathcal{V}$ denotes a token of the vocabulary $\mathcal{V}$, u_v the score the generator assigns it, and $T > 0$ the decoding temperature.

The probabilistic model

- $z \in D$ denotes a single retrieved document, and $\mathcal{Z}_K(q, D)$ the K documents with the highest retrieval score.
- $p_\eta(z \mid q, D)$ denotes the retrieval distribution with parameters η , and $p_\theta(y_i \mid q, C, y_{1:i-1}, S)$ the generator with parameters θ .
- $p_{\text{RAG}}(y \mid q, D, S)$ denotes the distribution over outputs.
- Y denotes the output viewed as a random variable, $M(Y)$ a measured property of it, and ε the randomness introduced by decoding.

The system prompt and its variation

- $\Sigma = \{\sigma_1, \dots, \sigma_m\}$ denotes the set of m requirements a system prompt states.
- φ denotes the wording in which those requirements are expressed, and $S = \rho(\Sigma, \varphi)$ denotes the resulting system prompt.
- $S_{b,0}, \dots, S_{b,3}$ denote the four prompt conditions of baseline $b \in \{A, B, C\}$, with $S_{b,0}$ the canonical form of that baseline, against which its three rewordings are compared. *Baseline* always refers to one of the three published prompts A , B and C .

Causal intervention

- $\text{do}(S = S_{b,j})$ denotes the intervention that sets the system prompt to condition $S_{b,j}$, and $\text{do}(C = c)$ the intervention that sets the context to c .

- $\tau_{b,j}$ denotes the average causal effect of condition $S_{b,j}$ relative to the canonical form $S_{b,0}$, taken over the query distribution P_Q , and $\hat{\tau}_{b,j}$ its sample estimate. Both are defined separately for each measure M and each evidence level e .
- $\mathbb{E}[\cdot]$ denotes expectation, and $\hat{\cdot}$ marks a quantity estimated from the sample.

Experimental design

- c_{ie} denotes the context given to query q_i at evidence level e , with $c_{i,\text{gold}}$ and $c_{i,\text{ret}}$ the two levels.
- $\mathcal{Q}_{\text{single}}$ and $\mathcal{Q}_{\text{multi}}$ denote the two query categories, and N_Q the size of the query set.
- A_q denotes the reference answers annotated for query q , against which correctness is measured.
- I denotes the set of queries in one cell of the design, and $\bar{M}_{ij}$ the mean of a measure over the n repetitions of query i under prompt condition j .
- The indices are $i \in \{1, \dots, N_Q\}$ over queries, $j \in \{0, \dots, 3\}$ over the wordings of a baseline, $e \in \{\text{gold}, \text{ret}\}$ over evidence levels, and $r \in \{1, \dots, n\}$ over the n repetitions of each cell. The index $k \in \{1, \dots, K\}$ is reserved for the rank of a retrieved passage.

2 Preliminaries

2.1 Path towards RAG

Large language models generate text from parametric knowledge alone, that is, from information encoded in their weights during training (Wang et al., 2024). This leads to two problems. First, a model’s knowledge is fixed at the time its training data was collected. Information that appears later is unknown to the model unless the model is retrained or fine-tuned. Second, purely parametric models are prone to hallucination: nothing in their design forces an output to trace back to a verifiable source, so they can generate plausible-sounding statements that are unsupported (Ji et al., 2023).

Both problems motivate giving a language model access to a knowledge base at inference time instead of relying only on what is stored in its parameters (Lin et al., 2025). Lewis et al. (2020) formalised this idea as RAG. RAG combines a retriever, which selects relevant passages from a document collection, with a pretrained generator, which conditions its output on those retrieved passages in addition to the input query.

2.2 Components of RAG

A RAG system has four components: the knowledge base the evidence comes from, the retriever that selects relevant evidence for a given query, the generator that produces a response from the query and the retrieved evidence, and the system prompt that specifies how the generator should behave. Chapter 4 treats them formally. Figure 2.1 shows how they fit together and which information passes between them.

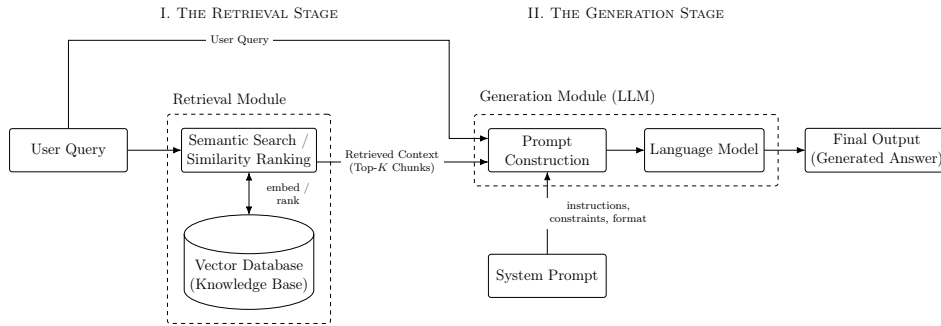


Figure 2.1: Architecture of a RAG system showing the four components introduced above. The retrieval stage matches the user query against a vector database and returns the top- K chunks. The generation stage combines those chunks with the user query and the system prompt into a single prompt and produces the answer.

2.2.1 Knowledge Base

The knowledge base $D = \{d_1, d_2, \dots, d_N\}$ is the collection of documents the evidence comes from. In most RAG systems, documents are first split into smaller passages, called chunks.

An embedding model maps each chunk to a high-dimensional vector, so that chunks with similar meaning lie close together in the embedding space. Similarity between a query vector and a chunk vector is

usually measured by the cosine of the angle between them.

The embeddings are stored in a vector database, which supports fast nearest-neighbour search over large collections of chunks. This thesis uses FAISS (Johnson et al., 2021), a library that is embedded directly in an application and gives fine-grained control over the index structure, because it supports the exact search that Section 5.6 requires.

2.2.2 Retriever

Given a user query q and a knowledge base D , the retriever selects the chunks that are relevant for answering q . Chapter 4 formalises this as a probability distribution $p_\eta(z \mid q, D)$ over the documents of the knowledge base.

2.2.2.1 Dense Retrieval

Dense retrieval embeds the query with the same embedding model used for the knowledge base and ranks chunks by the similarity between the query vector and each chunk vector. Because the embedding is learned, two passages that express the same content in different words lie close together in the embedding space. A query can therefore match a chunk with which it shares no vocabulary.

2.2.2.2 Retrieval Depth

The top- K ranked chunks form the retrieved context. The experiment of Chapter 5 fixes the retriever, the chunking and K , and controls the order of the passages.

2.2.3 Generator

The generator is the large language model that produces the final response. It generates text by autoregressive decoding. Its parameters, written θ in Chapter 4, encode what the model learned during training. This encoded knowledge is often called parametric memory.

The three inputs (the system prompt, the retrieved context and the query) reach the model in separate message roles. The system prompt is supplied in the system role, and the retrieved context and the query in the user role. The model’s chat template then assembles the messages into a single token sequence, with the system prompt first, so that it conditions the processing of every subsequent token. Chapter 4 formalises the generator as $p_\theta(y_i \mid q, C, y_{1:i-1}, S)$, the probability of the next token given the query, the retrieved context, the tokens already produced and the system prompt. Changing any one of these inputs can change the output, which is why the experiment varies only S .

2.2.3.1 Decoding and the Temperature Parameter

Generation is not deterministic by default. At each step the model produces a score u_v for every token v in the vocabulary $\mathcal{V}$. The softmax function, with a temperature parameter $T > 0$, turns these scores into a probability distribution:

$$p(v) = \frac{\exp(u_v/T)}{\sum_{v' \in \mathcal{V}} \exp(u_{v'}/T)}. \quad (2.1)$$

The temperature rescales the scores before they are normalised, and it controls how much probability mass is left for tokens the model considers less likely (Holtzman et al., 2020). As T approaches zero, the distribution concentrates on the highest-scoring token, and decoding

becomes effectively deterministic. As T grows, the distribution flattens.

This thesis holds the temperature T of Equation (2.1) fixed at the API default of 1.0 across all experimental conditions. The comparison between prompts requires only that T be constant.

2.2.4 System Prompt

The system prompt S specifies the instructions, constraints and output requirements the generator should follow. It has two functions. The first is to define the role the RAG system occupies, the audience it addresses, and the format its output takes. The second is to constrain the system’s behaviour; in a RAG system, the central constraint is the grounding constraint. A grounding constraint restricts the model to the retrieved context and specifies what to do when that context does not suffice.

2.3 Advanced RAG Architectures

The architecture described so far retrieves once per query and generates once from what was retrieved. Recent variants depart from this procedure and can be grouped by the stage of the pipeline they modify.

Conversational RAG. Storing prior turns of a dialogue as state, so that follow-up questions can be answered (Liu et al., 2024b).

Pre-retrieval optimisation. Rewriting or expanding the query before

it reaches the retriever.

Routing and adaptive retrieval. Deciding where and how much to retrieve.

Self-correction and reflection. Evaluating retrieved evidence before or during generation. Self-RAG instead trains the model to decide for itself when to retrieve and to critique its own draft output, so that retrieval and generation are interleaved (Asai et al., 2024).

Agentic RAG. Treating retrieval as one action among several available to an agent. A meta-agent delegates to document-specific or tool-specific agents and combines their results, which turns a fixed pipeline into a workflow whose steps are chosen at run time (Singh et al., 2025).

Each variant changes the retrieval function R , the generation function G , or the number of times they are applied. All of them, however, are still built from a retrieval step and a generation step.

This thesis implements only the simplest architecture and none of the variants above. The reason is methodological: the objective is to estimate the causal effect of the system prompt on the output, which Chapter 4 formalises as the intervention $\text{do}(S = S_{b,j})$. Because the prompt is set by intervention and everything else is held fixed, its total effect on the output can be estimated in any architecture. The simple architecture, however, makes the effect interpretable. In this architecture, the system prompt reaches the answer along a single path, through the generator, so the effect is an effect on generation. In the variants above the prompt, or the text it shapes, can also change what is retrieved, how often, or from where.

3 Literature Review

This chapter reviews two bodies of literature relevant to how the system prompt affects such a system (prompt sensitivity in language models, and retrieval-augmented generation itself) and then identifies the gap between them.

3.1 Prompt Sensitivity and Instruction-Following in Language Models

The first body of literature studies language models with no retrieval component. Cheng and Mastropaolo (2026) vary the system prompt given to instruction-tuned models for code generation, crossing prompt specificity and prompting strategy with model, programming language and decoding settings. They report that the system prompt influences the quality of generated code independently of the task description. Mu et al. (2025) examine how reliably models follow a system prompt when the user turn contains instructions conflicting with it, and report that the system prompt does not always prevail. Zheng et al. (2024) assign personas through the system prompt across a large set of roles and find that doing so does not reliably improve accuracy over a system prompt with no persona. Together, these three studies establish the system prompt as a channel with measurable influence over behaviour,

and show that the influence is not guaranteed.

A larger body of research varies the user-side prompt. Zhuo et al. (2024) propose ProSA, a framework that measures sensitivity to instance-level variation in the task instruction and relates it to decoding confidence. They report that larger models are more robust to such variation and that few-shot examples reduce sensitivity. Salinas and Morstatter (2024) perturb task prompts across text classification tasks (adding a trailing space, requesting the output in XML, or prefixing a jailbreak) and find that even such small perturbations change the labels the model assigns. Sclar et al. (2024) vary only the formatting of few-shot task templates (separators, casing and spacing, none of which carries propositional content) and report differences of up to 76 accuracy points between formats a reader would judge equivalent. Hua et al. (2025) evaluate twelve task templates across several benchmarks and argue that much of the measured sensitivity is produced by the evaluation method (log-likelihood scoring and rigid answer matching).

Ying et al. (2024) study conflict between a model’s parametric memory and the evidence supplied in its context, and classify models as intuitive or dependent according to which source they favour.

Two conclusions follow. The first, supported directly by the three system-prompt studies (Cheng and Mastropaolo, 2026; Mu et al., 2025; Zheng et al., 2024), is that the system prompt is a behavioural control variable whose effect can be isolated and measured in a properly controlled setup. The second is that model behaviour is sensitive to changes in the prompt that preserve its meaning. This holds whether the change is a rephrasing of the instruction (Zhuo et al., 2024), a perturbation as small as a trailing space (Salinas and Morstatter, 2024), or a change of formatting alone (Sclar et al., 2024). The apparent size of such an effect, however, depends on how it is measured (Hua et al.,

2025). This second conclusion is established for the user-side prompt, and its transfer to the system prompt is an assumption rather than an empirical finding. The assumption is reasonable, since both texts reach the same decoder as tokens in the same context window, but it is untested.

3.2 Retrieval-Augmented Generation: Architecture, Evaluation, and Failure Modes

The second body of literature concerns retrieval-augmented generation itself. Lewis et al. (2020) introduce RAG, combining a retriever with a pretrained sequence-to-sequence generator, and show that a generator conditioned on retrieved evidence outperforms purely parametric models on knowledge-intensive tasks. Their probabilistic formulation, in which retrieval is a distribution over documents and the retrieved document is marginalised out, is the direct basis for the model formalised in Chapter 4. Gao et al. (2023b) survey the development of RAG systems through Naive, Advanced and Modular paradigms. Their taxonomy treats retrieval and generation as separable modules, which allows this thesis to hold one fixed while studying an input to the other. Two later developments extend this taxonomy in directions this thesis deliberately does not follow. Edge et al. (2024) restructure the retrieval side into a knowledge graph, grouped into thematic segments and pre-summarised by an LLM, so that a query is answered by traversing entities and summaries. Singh et al. (2025) and Jin et al.

(2025) restructure the generation side into an agent loop, in which the model itself decides when and what to retrieve, calling R repeatedly and adaptively. Both directions relax exactly the components Chapter 4 holds fixed: R in the first case, and the single-call structure of G in the second. This thesis therefore retains Gao et al.’s simplest, Naive-RAG form, for the reason given in Section 2.3.

Es et al. (2023) propose Ragas, a reference-free evaluation framework for RAG pipelines built around three metrics that need no ground-truth human annotation: faithfulness, answer relevance and context relevance. This thesis adopts faithfulness and answer relevance. The third, context relevance, is a retrieval-side measure and is excluded for the reasons given in Section 5.7. Niu et al. (2024) introduce RAGTruth, a manually annotated corpus of word-level hallucinations produced by RAG systems, and benchmark hallucination-detection methods across multiple language models. Their corpus shows that RAG outputs contain claims the retrieved context does not support, which is the failure the faithfulness measure of this thesis captures.

Prompts have been compared within RAG research, but as a design choice rather than as the object of study Mao et al. (2024).

3.3 Synthesis and Research Gap

The first body of work shows two things: the system prompt is a controllable way of shaping language model behaviour, and model behaviour is sensitive to rewording that leaves the meaning of a prompt unchanged. Both are established without retrieval. The second body of work establishes the architecture, evaluation vocabulary and known

failure modes of RAG systems. Where it varies the prompt, it varies what the prompt asks for, in order to choose a better one.

The gap lies between the two. No study reviewed here holds a RAG pipeline fixed and varies only the wording of its system prompt while keeping its requirements constant. This thesis addresses this gap.

4 Formal Model of a RAG System

Chapter 3 identified the research gap: prompt sensitivity has so far been studied only outside retrieval, and RAG research varies prompts only to choose between them, not to measure the effect of their wording. Closing that gap requires a model of the system precise enough to isolate the system prompt. This chapter builds that model on the probabilistic formulation of RAG given by Lewis et al. (2020), adapts it to a generator that reads all retrieved passages at once, and makes the system prompt an explicit variable of the generation process.

4.1 Modelling Objective

The model can be read as a chain,

$$(q, D) \longrightarrow C \longrightarrow p_{\theta}(y_i \mid q, C, y_{1:i-1}, S) \longrightarrow p_{\text{RAG}}(y \mid q, D, S),$$

in which retrieval produces the context C and the system prompt S enters at the generation stage.

4.2 Probabilistic Retrieval

Retrieval is represented as a probability distribution over documents:

$$p_\eta(z \mid q, D). \quad (4.1)$$

Equation (4.1) denotes the probability that the retriever assigns to document z , given the query q and the knowledge base D . Lewis et al. (2020) obtain it by normalising the retrieval scores with a softmax over the documents. The formulation keeps only the K documents with the highest retrieval probability,

$$\mathcal{Z}_K(q, D) = \underset{z \in D}{\text{TopK}} \ p_\eta(z \mid q, D), \quad (4.2)$$

and the retrieved context passed to the generator is the resulting sequence of passages,

$$C = (z_1, \dots, z_K), \quad z_k \in \mathcal{Z}_K(q, D). \quad (4.3)$$

C is an ordered sequence, not a set, because the generator reads the passages in a particular order and that order can affect the answer (Liu et al., 2024a).

4.3 Prompt-Conditioned Generation

Lewis et al. (2020) write the generator at the level of a single token as $p_\theta(y_i \mid q, z, y_{1:i-1})$. It is the probability of the next token given the query, one retrieved document, and the tokens already produced. With the system prompt added as a further conditioning variable, this

becomes $p_\theta(y_i \mid q, z, y_{1:i-1}, S)$. Because generation is autoregressive, the probability of a complete output under document z follows by multiplication:

$$p_\theta(y \mid q, z, S) = \prod_{i=1}^L p_\theta(y_i \mid q, z, y_{1:i-1}, S). \quad (4.4)$$

4.4 From One Document to the Whole

Context

Equation (4.4) gives the probability of an output under a single document z . Since the system does not know which retrieved document supports the answer, Lewis et al. (2020) average over the retrieved documents, weighting each by its retrieval probability. They give two ways of doing this.

4.4.1 RAG-Sequence

RAG-Sequence assumes that a single retrieved document is responsible for the whole answer. The probability of an output is the weighted average, over documents, of the probability that document would produce that entire sequence:

$$p_{\text{RAG-Seq}}(y \mid q, D, S) \approx \sum_{z \in \mathcal{Z}_K(q, D)} p_\eta(z \mid q, D) \prod_{i=1}^L p_\theta(y_i \mid q, z, y_{1:i-1}, S). \quad (4.5)$$

4.4.2 RAG-Token

RAG-Token makes the weaker assumption that different parts of an answer may come from different documents. The average over documents is taken separately at every token:

$$p_{\text{RAG-Token}}(y \mid q, D, S) \approx \prod_{i=1}^L \sum_{z \in \mathcal{Z}_K(q, D)} p_{\eta}(z \mid q, D) p_{\theta}(y_i \mid q, z, y_{1:i-1}, S). \quad (4.6)$$

Both forms run the generator once per document. The system studied here does not. Like the VANILLA setting of Gao et al. (2023a), it concatenates the K passages into one prompt and runs the generator once, so that the model reads all of them together and decides for itself which to use. The average over documents is then no longer computed explicitly. It is replaced by a generator that conditions on the whole context C of Equation (4.3):

$$p_{\theta}(y_i \mid q, C, y_{1:i-1}, S). \quad (4.7)$$

Equation (4.7) defines the generator used in this thesis. It is here that the thesis departs from the original formulation in two ways: the model reads the whole context rather than one document, and the system prompt becomes part of what the generator conditions on. The distribution over complete outputs is

$$p_{\text{RAG}}(y \mid q, D, S) = \prod_{i=1}^L p_{\theta}(y_i \mid q, C(q, D), y_{1:i-1}, S), \quad (4.8)$$

where $C(q, D)$ is the context retrieved for q from D . Because the retriever is deterministic, the context is a function of the query and

the knowledge base. The only randomness left in Equation (4.8) is that of decoding.

4.5 The System Prompt: Requirements and Wording

The system prompt enters Equation (4.7) as a single symbol. It is, however, a text with internal structure, and the experiment varies its wording deliberately. A prompt states a set of requirements in a particular wording. Separating the two gives

$$S = \rho(\Sigma, \varphi), \quad \Sigma = \{\sigma_1, \dots, \sigma_m\}. \quad (4.9)$$

This formulation separates what a prompt asks for from how it is worded.

Three requirement sets are used. Each was taken from a prompt published in retrieval-augmented generation research and is reproduced in Section 5.3. Σ_A is the response-generation prompt of the RAGBench benchmark (Friel et al., 2024), which states a single requirement. Σ_B is the VANILLA instruction of Gao et al. (2023a), which comprises about ten clauses in four families (accuracy and concision, source restriction, citation, and tone). Of these, only citation can be checked by rule. Σ_C is the instruction of Mao et al. (2024), which states three.

The second component is the wording. For each requirement set, four wordings are written that are intended to hold Σ fixed and change only φ . The lexical and syntactic dimensions are taken from three

prior studies. Wahle et al. (2024) vary prompts by linguistically defined paraphrase type, and group those types into families, among them lexicon and syntax. Zhan et al. (2024) examine the lexical dimension by replacing individual words with semantically similar ones, and report that models are over-sensitive to variations undetectable to a reader. Kostić et al. (2026) separate a lexical pipeline built on synonym substitution from a syntactic one built on dependency parsing. They find that the two behave differently: lexical perturbation degrades performance consistently, while syntactic perturbation has more heterogeneous effects and occasionally improves results. The third dimension, layout, follows Sclar et al. (2024), who change only the formatting of a prompt.

This thesis follows their approach of varying the system prompt along linguistically defined, meaning-preserving dimensions one at a time. Writing $S_{b,j}$ for the j -th wording of baseline b :

$S_{b,0} = \rho(\Sigma_b, \varphi_0)$, **canonical**. The published form, unchanged.

$S_{b,1} = \rho(\Sigma_b, \varphi_1)$, **lexical**. Synonym substitution with clause order and voice held fixed.

$S_{b,2} = \rho(\Sigma_b, \varphi_2)$, **syntactic**. Clause reordering and conversion from active to passive voice.

$S_{b,3} = \rho(\Sigma_b, \varphi_3)$, **format**. The same wording laid out as a list rather than as prose.

Baseline A states the fewest requirements and is short enough to be given in full as an example. The four conditions read:

A0 Use the following pieces of context to answer the question.

```

A1  Employ the following excerpts of material to address the
      query.

A2  The question is to be answered using the context pieces
      below.

A3  CONTEXT: the following pieces
      TASK: use them to answer the question

```

Listing 4.1: The four conditions of baseline *A* from Friel et al. (2024).

The variants for baselines *B* and *C* are written on the same principle and are longer in proportion to their requirement sets. Section 5.3 gives them in full, together with the places where a rewording does not fully meet this principle.

4.6 The System Prompt as an Intervention

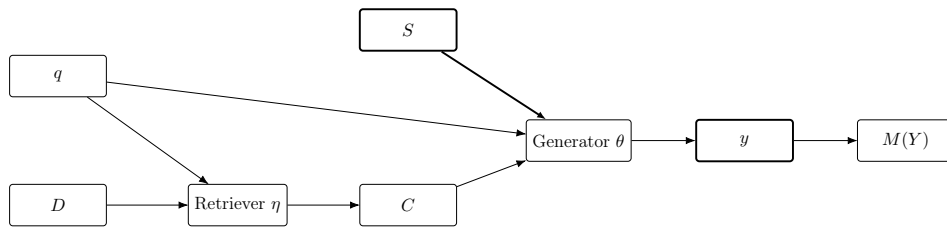


Figure 4.1: The causal structure of the system. The system prompt *S* and the output *y* are drawn in bold. *S* enters the generator and has no arrow into the retriever or the context *C*, so its effect on *y* runs through generation alone.

For the effect of the prompt to be interpretable, the following are held constant across conditions. Each query is used unchanged in every condition in which it is run, so that every comparison is made

within a query rather than between queries, and the knowledge base is fixed at $D = D_0$. The retriever and its parameters are fixed at $\eta = \eta_0$, and the language model at $\theta = \theta_0$.

Decoding is also stochastic, so the same configuration can produce different outputs from one run to the next. The term ε captures that variation:

$$Y = G(q, C, S; \theta, \varepsilon), \quad (4.10)$$

which is why each condition is repeated several times.

4.7 The Causal Effect

Setting the system prompt to a fixed value is an intervention, written $\text{do}(S = S_{b,j})$ following Pearl (2009). The notation separates *setting* a variable from merely *observing* it. For a query q under prompt condition $S_{b,j}$, the output of the system is the random variable

$$Y \sim p_{\text{RAG}}(y \mid q, D, S_{b,j}). \quad (4.11)$$

For one query the effect of the prompt is the difference in the expected value of the measured property $M(Y)$ of Equation (4.11) between the two conditions. The research question concerns the behaviour of the system on the queries it will actually encounter. The effect therefore has to be defined as an average over the distribution from which queries are drawn. Writing $P_{\mathcal{Q}}$ for that distribution over the query space $\mathcal{Q}$, the causal effect of prompt condition $S_{b,j}$ relative

to the canonical form $S_{b,0}$ of the same baseline is

$$\tau_{b,j} = \mathbb{E}_{q \sim P_Q} \left[\mathbb{E}_{Y \sim p_{\text{RAG}}(\cdot|q,D,S_{b,j})} [M(Y)] - \mathbb{E}_{Y \sim p_{\text{RAG}}(\cdot|q,D,S_{b,0})} [M(Y)] \right], \quad (4.12)$$

which connects the probability model of Section 4.4 directly to the quantity Chapter 5 estimates.

Equation (4.12) is an average treatment effect in the standard sense: the query plays the role of the unit, the prompt condition the role of the treatment, and $\tau_{b,j}$ the mean over units of the difference between the two treatments. The expectations are nested: the inner expectation averages over the randomness of decoding at a fixed query and a fixed context, while the outer averages over the queries themselves.

The distribution P_Q is not known and cannot be written down. Chapter 5 replaces it with the empirical distribution of a query set. The choice of data therefore determines how far the results generalise, which is why Section 4.8 draws the queries from an established question-answer dataset.

The choice of M is left open here. Section 4.9 describes what that property has to capture, and Section 5.7 fixes it.

4.8 Grounding the Knowledge Base in an Established Dataset

D and the query set are instantiated from an established question-answer dataset, MS MARCO (Nguyen et al., 2016). The corpus is public, so a third party can repeat the experiment, and its answers are scored with established metrics.

The collection is built from queries sampled from Bing search logs, with candidate passages retrieved for each query and free-form answers written by human judges. MS MARCO provides the queries q , the documents that populate D , and, through its annotations, the reference answers A_q and the passages the annotators marked as supporting them.

What it does not provide is a definition of a good answer.

4.9 What Counts as a Good Answer

Answer quality in a retrieval-augmented system comprises several properties that can fail independently of one another. The first is groundedness: an answer should rest on the documents the retriever actually supplied. The second is relevance: an answer should address the question that was asked, and address it directly. The third is correctness: an answer should agree with the reference answer recorded in the dataset, which is a different demand from the first and is treated separately here for the reason given below. The fourth is instruction adherence: whatever the system prompt asks for should be visible in the output itself, whether in length, in form, or in the handling of missing evidence. Instruction adherence is the property through which a prompt is meant to act, although a prompt can also change an answer in ways it did not ask for.

Groundedness and correctness are kept apart deliberately, because collapsing them would conceal the failure this thesis is concerned with. The retrieved context is neither complete nor guaranteed to be free of error. An answer can therefore be correct while ignoring the evidence

entirely, drawing instead on what the generator already holds in its parameters. It can also use the evidence faithfully and still be wrong, because the evidence was wrong. A high correctness score alone would reveal nothing about whether retrieval was used at all, and the use of retrieval is exactly what a system prompt is supposed to govern.

5 Experimental Setup

This chapter turns that model into a concrete experiment.

5.1 Research Question and Hypothesis

The experiment operationalises the research question of Chapter 1 as follows: *does a RAG system behave differently when the wording of its system prompt is changed while the meaning of the prompt stays the same?*

Wording refers to how the prompt is written: which words are used, how the sentences are built, and how the text is laid out. Meaning refers to the set of requirements Σ that the prompt states, in the sense of Equation (4.9). Every condition of the experiment keeps Σ fixed and changes only the wording φ . If the system responded only to what a prompt requires and not to how it is written, all wordings of one prompt would produce the same distribution of answers. Whether they do is the empirical question. Stated as hypotheses, it reads:

H_0 . System prompts that state the same requirements in different wordings produce the same distribution of answer quality: $\tau_{b,j} = 0$ for every rewording j , in every baseline, on every measure and at both evidence levels.

H_1 . At least one rewording changes answer quality: $\tau_{b,j} \neq 0$ for some rewording in some baseline on some measure at some evidence

level.

5.2 Overview of the Experiment

The system under test is the simple RAG architecture of Chapter 2. Table 5.1 lists the components of that system and the value each one is fixed at.

Table 5.1: The components of the system under test and their fixed values.

Component	Fixed value
Knowledge base D	passages of the 12,467 MS MARCO v2.1 validation queries with a well-formed answer, $N = 122,678$
Chunking χ	one passage per chunk, as published
Embedding model	all-mpnet-base-v2 (Reimers and Gurevych, 2019), mean pooling, L2-normalised
Vector index	FAISS flat inner-product index, exact search
Retrieval depth K	10
Generator θ	gpt-4o-mini (OpenAI, 2024) via the chat completions API
Decoding	temperature 1.0 (the API default)
Judge	gpt-4.1-mini (OpenAI, 2025), temperature 0, fixed prompts
Query set	100 MS MARCO queries, seed 20260904
Repetitions n	3 per cell

The experiment has three stages, of which the first two collect data. In the first stage, retrieval is run once for every query and its result is written to disk. In the second stage, every prompt condition is applied to every query using the stored retrieval results, and the answers are written to disk. In the third stage, the answers are scored and the prompt effect is estimated.

5.3 System-Prompt Conditions

Baseline *A* is the response-generation prompt of Friel et al. (2024), reproduced in Listing 5.1. It states one requirement: answer from the supplied context.

```
Use the following pieces of context to answer the question.
{documents}
Question: {question}
```

Listing 5.1: Baseline *A*: the response-generation prompt of Friel et al. (2024).

Baseline *B* is the VANILLA instruction of Gao et al. (2023a). It constrains accuracy, engagement, concision and register, it fixes the syntax and the density of citations, and it warns that some of the supplied results may be irrelevant.

```
Write an accurate, engaging, and concise answer for the given
question using only the provided search results (some of
    which
might be irrelevant) and cite them properly. Use an unbiased
    and
journalistic tone. Always cite for any factual claim. When
    citing
several search results, use [1][2][3]. Cite at least one
    document
and at most three documents in each sentence. If multiple
    documents
support the sentence, only cite a minimum sufficient subset
    of the
documents.
```

Listing 5.2: Baseline *B*: the VANILLA instruction of Gao et al. (2023a), reproduced from their Table 23.

Baseline *C* is the instruction of Mao et al. (2024). Its authors describe it as consisting of three parts: the first directs the model to the retrieved passages, the second asks it to read the question carefully, and the third asks for an answer together with a justification.

```

Refer to the passage below and answer the following question.
Make sure you fully understand the meaning of the question
    and
passages. Then give the answer and explain why you choose
    this
answer.
Passages: {passages}
Question: {question}

```

Listing 5.3: Baseline *C*: the comprehensive instruction of Mao et al. (2024).

The published prompts contain placeholders for the documents and the question. In this experiment the instruction text alone forms the system prompt *S* and is sent in the system role of the chat interface. The passages and the question are sent in the user role, in the fixed template of Listing 5.4. The template is the same in every condition, so the only text that differs between conditions is *S*. Passages are numbered from one, so that citations can refer to them and the position of a passage can be recovered from the answer.

```

Passages:
[1] <first passage>
[2] <second passage>

```

```

...

Question: <the query>
Answer:

```

Listing 5.4: The user turn. It is identical in every condition.

A rewording that changed how strongly an instruction was stated, for example by turning *always cite* into *please try to cite*, would change what the prompt requires. Such a change is a change of Σ , and a difference in behaviour caused by it could not be read as an effect of wording. The rewordings were written to avoid such changes. In four places, however, they come close. These are listed here so that the results can be interpreted with them in mind:

- C1 asks the model to *justify why you select* its answer where C0 asks it to *explain why you choose* it. *Justify* may ask for slightly more than *explain*.
- C3 adds *and nothing else* to the source line, which states an exclusivity that C0 leaves implicit.
- B2 and C2 turn several instructions into plain statements (*A citation is always given*, *The answer is then given*) rather than obligations (*is to be given*), which may state them less strongly.
- B3 and C3 keep some items as imperatives (*cite the search results properly*, *give the answer*), so the list format does not eliminate every imperative.

The four wordings of baseline *A* are given in Listing 4.1 in Chapter 4. Listings 5.5 and 5.6 give the three rewordings of baselines *B* and *C*; their canonical forms are Listings 5.2 and 5.3 above.

- B1 Compose a precise, compelling, and succinct reply for the posed query drawing only on the supplied search results (a number of which may be immaterial) and reference them correctly. Adopt an impartial and journalistic register. Invariably reference every factual assertion. When referencing several search results, employ [1] [2] [3]. Reference at minimum one document and at maximum three documents per sentence. If several documents corroborate the sentence, only reference a minimal adequate subset.
- B2 For the given question an accurate, engaging, and concise answer is to be written from the provided search results alone, some of which might be irrelevant, and cited properly. An unbiased, journalistic tone is to be used. A citation is always given for any factual claim. Where several results are cited, [1] [2] [3] is used. In each sentence at least one and at most three documents are cited. Where multiple documents support a sentence, only a minimum sufficient subset is cited.
- B3 ANSWER
- accurate, engaging, and concise, for the given question

```

- source: only the provided search results
- note: some of the provided results might be irrelevant

CITATIONS

- cite the search results properly
- always cite for any factual claim
- when citing several search results: [1][2][3]
- in each sentence: at least one document, at most three
documents
- if multiple documents support the sentence: only a
minimum
  sufficient subset

TONE

- unbiased and journalistic

```

Listing 5.5: The lexical ($B1$), syntactic ($B2$) and format ($B3$) wordings of baseline B .

```

C1 Consult the excerpt below and respond to the following
query.

Ensure you completely grasp the sense of the query and
excerpts.

Then supply the response and justify why you select this
response.

C2 The following question is to be answered from the passage
below.

The meaning of the question and passages is first
understood.

The answer is then given, with an explanation of why it
is
chosen.

```

C3	SOURCE: the passage below, and nothing else
	READ: fully understand the meaning of the question and passages
	ANSWER: give the answer to the following question
	EXPLAIN: say why you choose this answer

Listing 5.6: The lexical ($C1$), syntactic ($C2$) and format ($C3$) wordings of baseline C .

Prompt length is held nearly constant within a baseline, so that a difference between two wordings cannot be an effect of how much text the prompt contains. Table 5.2 gives the word counts. No wording departs from its canonical form by more than one tenth.

Table 5.2: Word count of every condition, with the deviation from the canonical form of its baseline.

Baseline	canonical	lexical	syntactic	format
A	10	10 ($\pm 0\%$)	11 ($+10\%$)	11 ($+10\%$)
B	74	71 (-4%)	80 ($+8\%$)	77 ($+4\%$)
C	33	32 (-3%)	34 ($+3\%$)	32 (-3%)

This yields twelve conditions: three baselines in four wordings each. They are written $S_{b,j}$ with $b \in \{A, B, C\}$ and $j \in \{0, 1, 2, 3\}$. Comparing $S_{B,1}$ with $S_{A,0}$ would mix a change of wording with a change of what is asked for. Hence, each rewording is compared only with the canonical form of its own baseline.

5.4 Variables and Controlled Factors

Independent variable. The wording j of the system prompt for each baseline b .

Dependent variable. The quality of the answer, as scored by the measures of Section 5.7: faithfulness, answer relevance, correctness and instruction adherence, with correctness measured by three metrics: ROUGE-L, BLEU-1 and BERTScore. Each of the six is used in turn as the measured property M of Equation (4.12), so the effect of a rewording is estimated six times. All six take part in the test of H_0 . They are reported separately.

Controlled factor. The context C passed to the generator is set directly, as the intervention $\text{do}(C = c_{ie})$ of Section 4.4, at two evidence levels e :

$c_{i,\text{gold}}$. Only the passages the annotation marks as supporting the answer. The evidence needed for the answer is guaranteed to be present.

$c_{i,\text{ret}}$. The K passages the fixed retriever actually returns.

The two levels differ in more than the quality of the evidence. A gold context holds one or a few passages, a retrieved context always ten.

Held constant. Everything in Table 5.1. Queries and passages are embedded separately by the same model and compared by their vectors, as in the bi-encoder of Karpukhin et al. (2020), but with one shared encoder for both sides (Reimers and Gurevych, 2019). Because the embedding model, the index, the chunking and the depth are all fixed, the set $\mathcal{Z}_K(q, D)$ of Equation (4.2) depends on the query alone and never on the system prompt.

Covariates. Two properties of each query are recorded: the category (single-passage or multi-passage) and MS MARCO’s intent type. Both are defined in Section 5.5. They are balanced in the sample so that

it covers different kinds of query. The analysis of this thesis does not break the effects down by them.

The design. An observation is one answer, written

$$y_{ijer} \sim p_{\text{RAG}}(y \mid q_i, \text{do}(C = c_{ie}), \text{do}(S = S_{b,j})), \quad r = 1, \dots, n, \quad (5.1)$$

for query i , prompt condition $S_{b,j}$, evidence level e and repetition r . The twelve prompt conditions and the two evidence levels are fully crossed, which gives 24 cells. With 100 queries and $n = 3$ repetitions this would be 7,200 generations. Queries that fail the retrieval gate of Section 5.6 are not run at the retrieved level, so fewer answers are generated (Chapter 6). Within a cell the only remaining source of variation is the decoding randomness ε of Equation (4.10). Each repetition is an independent draw from the same distribution.

5.5 Knowledge Base and Query Set

Following Section 4.8, both the knowledge base and the query set come from MS MARCO (Nguyen et al., 2016), read from the Hugging Face export of version 2.1. It holds 101,093 queries, of which 12,467 carry a non-empty `wellFormedAnswers` field. Only those rows are used, because the well-formed answers are the references against which correctness is measured.

The knowledge base. The corpus holds every passage attached to any of the 12,467 rows. MS MARCO provides at most ten candidate passages per query, retrieved by Bing and then judged by annotators.

The corpus therefore contains the gold evidence for every query. Each passage forms one chunk and is used unchanged.

The query set. A total of $N_Q = 100$ queries are sampled from the same 12,467 rows. They fall into two categories. A row is *single-passage* when exactly one of its passages is marked `is_selected`, and *multi-passage* when two or more are. Fifty queries are drawn from each category.

Within each category the draw is balanced across MS MARCO’s five intent types, DESCRIPTION, ENTITY, LOCATION, NUMERIC and PERSON, ten queries per type. The distribution within MS MARCO is uneven: on the well-formed rows DESCRIPTION makes up 44% and PERSON 7%. Of the 12,467 well-formed rows, 12,062 are single-passage and only 386 multi-passage. The remaining 19 rows have no passage marked and are not sampled.

5.6 Experimental Procedure

5.6.1 Stage 1: Retrieval and the Retrieval Gate

Queries and passages are encoded with `all-mpnet-base-v2`, using mean pooling over the last layer and L2 normalisation. For normalised vectors, the inner product therefore equals the cosine similarity.

The retrieval stage is also where the evidence is checked. A query passes the *retrieval gate* when every passage its annotation marks as supporting appears among the $K = 10$ returned. Table 5.3 reports the result.

Queries that fail the gate are excluded from the retrieved level. The

Table 5.3: The retrieval gate at the depth used, $K = 10$. A query passes when all of its gold passages are among the ten returned.

K	Passed	single-passage	multi-passage
10 (used)	88 / 100	45 / 50	43 / 50

measures of Section 5.7 cannot be meaningfully interpreted when the relevant evidence was never supplied. When the context lacks the passage carrying the answer, a correct answer can only have come from the generator’s memory. An incorrect answer cannot be told apart from a faithful reading of the other passages.

5.6.2 Stage 2: Generation

For every query, every cell of the design is run $n = 3$ times, except the retrieved-level cells of queries that failed the gate. The cells are visited in a random order per query. Every generation is an independent request.

The order of the passages within the context is shuffled, because answer accuracy depends on where the relevant passage is located (Liu et al., 2024a). Shuffling randomises the position of the gold passages across conditions, so that position cannot systematically favour any condition.

5.7 Evaluation

The evaluation scores each answer on the four properties of Section 4.9.

Each property is estimated by one measure, except correctness,

which is estimated by three. The first three are the measures that the survey of Yu et al. (2024) names as the evaluation targets of the generation component: faithfulness relates the response to the retrieved documents, answer relevance relates it to the query, and correctness relates it to the reference answer. Retrieval measures are excluded by construction, because the system prompt cannot change what is retrieved.

5.7.1 Six Measures for Four Properties

Faithfulness. Whether every claim in the answer can be traced to a supplied passage. It is scored reference-free, after Es et al. (2023), in two steps. First the judge (Section 5.7.2) breaks the answer into atomic factual claims. Then, for each claim, the judge decides whether the passages state it explicitly.

Answer relevance. Whether the answer addresses the question actually asked. A prompt can degrade an answer without making it false. Relevance is scored reference-free, following Es et al. (2023): the judge generates three questions to which the answer would be a suitable response. The score is the mean cosine similarity between these questions and the original question, computed in the embedding space of the retrieval encoder.

Correctness. Whether the answer agrees with the reference answers A_q . It is measured with the two metrics that MS MARCO uses for its free-form answers (Nguyen et al., 2016) and that Lewis et al. (2020) report on this task: ROUGE-L (Lin, 2004) and BLEU-1 (Papineni et al., 2002). Both compare the answer with the references word by word.

Before scoring, the text is lowercased and punctuation is replaced by spaces. Citation markers such as [3] are also removed from the answer. Baseline *B* asks for them, and they would otherwise penalise its answers for following the prompt. ROUGE-L is the F-measure based on the longest common subsequence of the answer and the reference. BLEU-1 is the share of the answer’s words that also occur in a reference. Each word is counted at most as often as it appears in any single reference. The result is multiplied by a brevity penalty when the answer is shorter than the reference closest to it in length. The third metric, BERTScore (Zhang et al., 2020), embeds the answer and the reference with a pretrained encoder, matches every word of one to the most similar word of the other by the cosine of their embeddings, and combines the two directions into an F1 score. It is computed with the standard settings for English (Liu et al., 2019).

Instruction adherence. Whether the answer complies with the instructions in the prompt. It is scored as strict accuracy in the sense of Zhou et al. (2023): an answer is adherent only if it satisfies every checked requirement. The score is binary.

5.7.2 The Judge

Faithfulness and answer relevance have no reference string. Therefore, they are scored by a second language model, as Es et al. (2023) and Saad-Falcon et al. (2024) do. The judge is `gpt-4.1-mini`, a different model from the generator, so that the generator never grades its own output. It is run at temperature 0 so that the verdict depends as far as possible on the answer and the passages and not on a random draw. Zheng et al. (2023) document several biases of language-model judges,

among them a preference for longer answers.

5.7.3 Estimating the Effect of a Rewording

The quantity estimated is the average treatment effect of Equation (4.12).

Here, M is instantiated by each of the six measures in turn. Every measure is oriented so that a higher value is better. A positive effect means the rewording improved that measure.

For one measure M , one evidence level e and one baseline b , the estimate of the effect of wording j is

$$\hat{\tau}_{b,j} = \frac{1}{|I|} \sum_{i \in I} (\bar{M}_{ij} - \bar{M}_{i0}), \quad \bar{M}_{ij} = \frac{1}{n} \sum_{r=1}^n M(y_{ijer}), \quad (5.2)$$

where I is the set of queries and $\bar{M}_{ij}$ is the mean of the measure over the n repetitions of query i under condition $S_{b,j}$. Equation (5.2) is the sample version of Equation (4.12). The inner average estimates the expectation over decoding, and the outer one the expectation over queries. The repetitions are averaged first, so that each query contributes one difference.

6 Results and Discussion

The run produced 6,768 answers. At the gold level all 100 queries are usable, which gives $100 \times 12 \times 3 = 3,600$ answers. At the retrieved level 88 of the 100 queries pass the gate of Section 5.6 at $K = 10$, which gives $88 \times 12 \times 3 = 3,168$.

6.1 Descriptive Results

Table 6.1 gives the mean of each measure in each cell, and Table 6.2 the mean length of the answers.

The measures differ sharply in how sensitive they are to the prompt. Faithfulness is between 0.912 and 0.976 everywhere. Relevance is stable, between 0.693 and 0.746. Adherence runs from 0.083 to 0.943. ROUGE-L runs from 0.201 to 0.550, BLEU-1 from 0.119 to 0.505 and BERTScore from 0.276 to 0.610. All three are highest under baseline *A*, whose answers are the shortest, and lowest under baseline *C*, whose answers are the longest, because they must include a justification.

Faithfulness is higher at the retrieved level than at the gold level in every condition, although the gold passages are the better evidence.

Table 6.1: Mean of each measure per condition. The gold level rests on 100 queries and the retrieved level on the 88 that pass the gate at $K = 10$. Correctness is given as ROUGE-L, BLEU-1 and BERTScore. Adherence is undefined for baseline A, whose only requirement is grounding.

	Faithful.	Relevance	ROUGE-L	BLEU-1	BERTScore	Adherence
<i>gold</i> , $ I = 100$						
A0	0.917	0.736	0.541	0.494	0.597	n/a
A1	0.922	0.730	0.519	0.463	0.583	n/a
A2	0.930	0.734	0.550	0.505	0.610	n/a
A3	0.930	0.741	0.539	0.492	0.603	n/a
B0	0.912	0.696	0.362	0.285	0.440	0.223
B1	0.942	0.705	0.407	0.332	0.470	0.447
B2	0.924	0.703	0.397	0.319	0.464	0.343
B3	0.931	0.703	0.403	0.327	0.473	0.297
C0	0.924	0.729	0.262	0.164	0.358	0.943
C1	0.919	0.738	0.272	0.174	0.359	0.940
C2	0.941	0.728	0.296	0.193	0.387	0.853
C3	0.939	0.731	0.322	0.214	0.395	0.923
<i>retrieved</i> , $K = 10$, $ I = 88$						
A0	0.968	0.737	0.436	0.377	0.497	n/a
A1	0.976	0.734	0.418	0.359	0.481	n/a
A2	0.974	0.732	0.443	0.383	0.508	n/a
A3	0.976	0.736	0.438	0.376	0.499	n/a
B0	0.962	0.693	0.233	0.157	0.325	0.083
B1	0.971	0.705	0.297	0.220	0.369	0.345
B2	0.957	0.696	0.267	0.192	0.349	0.193
B3	0.965	0.702	0.282	0.205	0.358	0.205
C0	0.963	0.734	0.201	0.119	0.276	0.913
C1	0.973	0.738	0.207	0.125	0.279	0.932
C2	0.973	0.746	0.229	0.141	0.306	0.788
C3	0.959	0.730	0.267	0.170	0.337	0.920

Table 6.2: Mean answer length in tokens per condition.

	A0	A1	A2	A3	B0	B1	B2	B3	C0	C1	C2	C3
gold	41	44	38	39	67	59	62	60	96	93	84	68
retrieved	67	80	64	68	112	93	104	99	140	136	119	88

6.2 The Effect of Rewording

In the tables that follow, a star marks an estimate that is significant after the Holm adjustment.

6.2.1 Instruction Adherence

The largest effects appear on this measure. Table 6.3 gives all six estimates at both evidence levels, for baselines *B* and *C*.

Table 6.3: Effect of each rewording on instruction adherence, against the canonical form of its own baseline. A star marks a Holm-adjusted *p*-value below 0.05.

Rewording	gold		retrieved, $K = 10$	
	$\hat{\tau}$	95% CI	$\hat{\tau}$	95% CI
B1 lexical	+0.223*	[+0.158, +0.289]	+0.261*	[+0.187, +0.336]
B2 syntactic	+0.120*	[+0.071, +0.169]	+0.110*	[+0.045, +0.175]
B3 format	+0.073*	[+0.026, +0.121]	+0.121*	[+0.058, +0.185]
C1 lexical	−0.003	[−0.039, +0.032]	+0.019	[−0.020, +0.058]
C2 syntactic	−0.090*	[−0.140, −0.040]	−0.125*	[−0.187, −0.063]
C3 format	−0.020	[−0.067, +0.027]	+0.008	[−0.048, +0.063]

Eight of the twelve estimates are significant. Every rewording of baseline *B* raises adherence at both evidence levels. Of the rewordings of *C*, only the syntactic one, C2, has a significant effect. C1 and C3 have no significant effect.

Regarding *B* the rule fails an answer when any of its sentences carries no citation. Under the canonical wording B0 the first sentence of an answer is uncited in 77% of the gold answers and in 90% of the retrieved ones. Under B1 these shares fall to 53% and 63%.

For baseline *C*, the rewording that lowers adherence is the one that turns the instructions into plain passive statements, such as *The answer is then given, with an explanation of why it is chosen*. A plausible

explanation is that it reads as a description rather than a request, and is therefore followed less often. This fits the concern raised in Section 5.3, that such statements may state a requirement less strongly than the canonical imperative.

6.2.2 Faithfulness

One of the nine estimates at the gold level is significant, namely B1, $\hat{\tau} = +0.031$ with a 95% interval of $[+0.014, +0.047]$. The same comparison at the retrieved level gives $+0.009$ with an interval that contains zero. A result that appears at one level and not the other is weak evidence.

6.2.3 Answer Relevance

No estimate is significant at either level. The largest estimate in absolute value is $+0.012$.

6.2.4 Correctness

Tables 6.4, 6.5 and 6.6 give the estimates.

Every rewording of baseline B raises correctness. Of the rewordings of C , the syntactic and format ones raise it. The format one raises it most. No estimate for the lexical rewording C1 is significant. For baseline A , the lexical rewording A1 lowers it. The BLEU-1 estimate there is not significant after adjustment. Every significant BERTScore effect has the same direction as the ROUGE-L and BLEU-1 effects of

Table 6.4: Effect of each rewording on correctness measured by ROUGE-L, against the canonical form of its own baseline. A star marks a Holm-adjusted p -value below 0.05.

Rewording	gold		retrieved, $K = 10$	
	$\hat{\tau}$	95% CI	$\hat{\tau}$	95% CI
A1 lexical	-0.022*	[-0.036, -0.008]	-0.018*	[-0.031, -0.005]
A2 syntactic	+0.009	[-0.002, +0.021]	+0.008	[-0.006, +0.022]
A3 format	-0.002	[-0.016, +0.012]	+0.002	[-0.007, +0.011]
B1 lexical	+0.045*	[+0.027, +0.062]	+0.064*	[+0.042, +0.087]
B2 syntactic	+0.034*	[+0.020, +0.049]	+0.034*	[+0.016, +0.051]
B3 format	+0.041*	[+0.027, +0.054]	+0.049*	[+0.028, +0.070]
C1 lexical	+0.010	[-0.000, +0.020]	+0.006	[-0.002, +0.013]
C2 syntactic	+0.034*	[+0.025, +0.043]	+0.027*	[+0.020, +0.034]
C3 format	+0.060*	[+0.047, +0.072]	+0.066*	[+0.054, +0.077]

Table 6.5: Effect of each rewording on correctness measured by BLEU-1, against the canonical form of its own baseline. A star marks a Holm-adjusted p -value below 0.05.

Rewording	gold		retrieved, $K = 10$	
	$\hat{\tau}$	95% CI	$\hat{\tau}$	95% CI
A1 lexical	-0.031*	[-0.049, -0.013]	-0.017	[-0.031, -0.003]
A2 syntactic	+0.011	[-0.003, +0.025]	+0.007	[-0.009, +0.022]
A3 format	-0.002	[-0.020, +0.015]	-0.001	[-0.011, +0.009]
B1 lexical	+0.047*	[+0.027, +0.068]	+0.063*	[+0.037, +0.089]
B2 syntactic	+0.034*	[+0.018, +0.050]	+0.035*	[+0.016, +0.054]
B3 format	+0.042*	[+0.026, +0.058]	+0.048*	[+0.028, +0.069]
C1 lexical	+0.010	[+0.002, +0.018]	+0.006	[+0.001, +0.010]
C2 syntactic	+0.029*	[+0.022, +0.037]	+0.022*	[+0.017, +0.027]
C3 format	+0.050*	[+0.041, +0.059]	+0.051*	[+0.044, +0.058]

Table 6.6: Effect of each rewording on correctness measured by BERTScore, against the canonical form of its own baseline. A star marks a Holm-adjusted p -value below 0.05.

Rewording	gold		retrieved, $K = 10$	
	$\hat{\tau}$	95% CI	$\hat{\tau}$	95% CI
A1 lexical	-0.014^*	$[-0.024, -0.004]$	-0.016^*	$[-0.027, -0.005]$
A2 syntactic	$+0.013^*$	$[+0.004, +0.022]$	$+0.011$	$[+0.000, +0.021]$
A3 format	$+0.006$	$[-0.004, +0.015]$	$+0.002$	$[-0.007, +0.011]$
B1 lexical	$+0.030^*$	$[+0.017, +0.044]$	$+0.044^*$	$[+0.027, +0.062]$
B2 syntactic	$+0.024^*$	$[+0.013, +0.035]$	$+0.024^*$	$[+0.011, +0.037]$
B3 format	$+0.033^*$	$[+0.022, +0.044]$	$+0.033^*$	$[+0.018, +0.048]$
C1 lexical	$+0.002$	$[-0.006, +0.009]$	$+0.003$	$[-0.005, +0.011]$
C2 syntactic	$+0.030^*$	$[+0.021, +0.038]$	$+0.030^*$	$[+0.021, +0.038]$
C3 format	$+0.037^*$	$[+0.025, +0.049]$	$+0.061^*$	$[+0.047, +0.076]$

that rewording.

These effects track answer length. Every rewording that raised correctness produced shorter answers than its canonical form (Table 6.2), and A1, which lowered it, produced longer ones. A2, which raised BERTScore, produced slightly shorter answers as well. Across the eighteen comparisons, the change in mean length and the change in ROUGE-L have a correlation of -0.79 , and for BERTScore the correlation is -0.87 . The reason is that the references are short, so a shorter answer shares a larger fraction of its words, and of its meaning, with them. BERTScore is also affected, because its precision component penalises an answer for everything it says beyond the reference. The recall component of ROUGE-L alone, which measures how much of the reference an answer contains and does not penalise length, shows no significant gain for any rewording. The rewordings therefore changed how closely the answers matched the reference mainly by changing their length, not their content.

6.3 The Verdict on the Hypothesis

One hundred and two estimates were made across the six measures of Section 5.7, and forty-five of them are significant after adjustment: eight on adherence, thirty-six on the three correctness metrics and one on faithfulness. H_0 is therefore rejected. H_1 , that at least one rewording changes answer quality, is supported: seven of the nine rewordings have a significant effect on at least one measure. A3 and C1 have no significant effect on any measure.

The rejection does not rest on the single faithfulness result. It rests on instruction adherence, where four rewordings have clear effects, three positive and one negative, and on correctness, where the effects are small and track answer length.

6.4 Limitations

Four limitations apply to these results. The first is the collection. Every figure is conditional on MS MARCO. The effects therefore suggest, but do not establish, what may be expected on other data. The second concerns correctness. ROUGE-L and BLEU-1 reward word overlap and BERTScore semantic similarity. None of them measures factual accuracy, and all three favour answers close in length to the reference. The third concerns the rewordings themselves. Four places where a rewording may have changed how strongly a requirement is stated were named in Section 5.3. The effect of C2 in particular may partly reflect a weaker statement of the requirement rather than a change of wording alone. The fourth concerns the judge: faithfulness and answer relevance are scored by a language model, which may be

biased, for example towards longer answers (Section 5.7.2).

7 Conclusion and Future Work

7.1 Conclusion

This thesis asked to what extent systematic changes in the system prompt influence the behaviour and outcomes of a RAG system when the user input, the knowledge base, the retrieval mechanism and the underlying language model are held fixed. It answered the question for one kind of change, a change of wording that leaves the requirements of the prompt unchanged.

The answer is that wording matters. It mattered most for whether the answers complied with the prompt. Rewording the citation instruction of Gao et al. (2023a) raised the share of fully cited answers. Turning the instruction of Mao et al. (2024) into passive statements lowered the share of justified answers. Wording also changed the length of the answers, and thereby their agreement with the reference answer. It had no consistent effect on whether the answers stayed within the supplied passages or addressed the question. These results held at both evidence levels, that is, both with only the supporting passages and with the actual output of the retriever.

The contribution of this thesis: The formal model of Chapter 4 makes the system prompt an explicit variable of the generator and defines its effect as an average treatment effect over queries. The design of Chapter 5 separates what a prompt asks for from how it

is worded, fixes everything else, sets the evidence directly, and pairs every comparison within a query. The same design can be reused with other prompts, models and datasets.

7.2 Future Work

More generators. Every result here comes from one model, `gpt-4o-mini`. Whether other models respond to the same rewordings in the same way remains an open question. Repeating the experiment with a second API model and an open model of a different size would show whether the effects are specific to the model or arise from the wording itself.

Correctness beyond overlap. The three correctness metrics compare wording and meaning with a reference but do not verify factual accuracy. A judge that decides whether an answer contains the fact stated in the reference, validated against human labels on a sample, would measure correctness independently of answer length.

Other data. A dataset with longer answers, and an open collection in which the answer may be missing, would test whether the effects hold where retrieval is harder and where the prompt has to govern what the model does when the evidence does not suffice.

Bibliography

- Asai, A., Wu, Z., Wang, Y., Sil, A., and Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*.
- Cheng, Z. and Mastropaolo, A. (2026). An empirical study on the effects of system prompts in instruction-tuned models for code generation.
- Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., and Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization.
- Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2023). Ragas: Automated evaluation of retrieval augmented generation.
- Friel, R., Belyi, M., and Sanyal, A. (2024). RAGBench: Explainable benchmark for retrieval-augmented generation systems.
- Gao, T., Yen, H., Yu, J., and Chen, D. (2023a). Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. (2023b). Retrieval-augmented generation for large language models: A survey.

- Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. (2020). The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*.
- Hua, A., Tang, K., Gu, C., Gu, J., Wong, E., and Qin, Y. (2025). Flaw or artifact? rethinking prompt sensitivity in evaluating LLMs. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.
- Jin, B., Zeng, H., Yue, Z., Yoon, J., Arik, S., Wang, D., Zamani, H., and Han, J. (2025). Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning.
- Johnson, J., Douze, M., and Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781. Association for Computational Linguistics.
- Kostić, B., Fallon, C., Risch, J., and Löser, A. (2026). Same meaning, different scores: Lexical and syntactic sensitivity in LLM evaluation. To appear in the Proceedings of the 2026 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC).

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474.
- Lin, C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, Barcelona, Spain. Association for Computational Linguistics.
- Lin, Y.-H., Chen, Q.-H., Cheng, Y.-J., Zhang, J.-R., Liu, Y.-H., Hsia, L.-Y., and Chen, Y.-N. (2025). LLM inference enhanced by external knowledge: A survey.
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024a). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach.
- Liu, Z., Ping, W., Roy, R., Xu, P., Lee, C., Shoeybi, M., and Catanzaro, B. (2024b). ChatQA: Surpassing GPT-4 on conversational QA and RAG. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.
- Mao, Y., Dong, X., Xu, W., Gao, Y., Wei, B., and Zhang, Y. (2024). FIT-RAG: Black-box RAG with factual information and token reduction.

- Mu, N., Lu, J., Lavery, M., and Wagner, D. A. (2025). A closer look at system prompt robustness.
- Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., and Deng, L. (2016). MS MARCO: A human generated MACHine Reading COmprehension dataset.
- Niu, C., Wu, Y., Zhu, J., Xu, S., Shum, K., Zhong, R., Song, J., and Zhang, T. (2024). RAGTruth: A hallucination corpus for developing trustworthy retrieval-augmented language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*.
- OpenAI (2024). GPT-4o mini: Advancing cost-efficient intelligence. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>.
- OpenAI (2025). Introducing GPT-4.1 in the API. <https://openai.com/index/gpt-4-1/>.
- Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. (2002). BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA. Association for Computational Linguistics.
- Pearl, J. (2009). *Causality: Models, Reasoning, and Inference*. Cambridge University Press, Cambridge, 2 edition.
- Reimers, N. and Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*

and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pages 3982–3992.

- Saad-Falcon, J., Khattab, O., Potts, C., and Zaharia, M. (2024). ARES: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 338–354.
- Salinas, A. and Morstatter, F. (2024). The butterfly effect of altering prompts: How small changes and jailbreaks affect large language model performance. In *Findings of the Association for Computational Linguistics: ACL 2024*.
- Sciar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. (2024). Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*.
- Singh, A., Ehtesham, A., Kumar, S., Khoei, T. T., and Vasilakos, A. V. (2025). Agentic retrieval-augmented generation: A survey on agentic RAG.
- Wahle, J. P., Ruas, T., Xu, Y., and Gipp, B. (2024). Paraphrase types elicit prompt engineering capabilities. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 11004–11033, Miami, Florida, USA. Association for Computational Linguistics.
- Wang, M., Yao, Y., Xu, Z., Qiao, S., Deng, S., Wang, P., Chen, X., Gu, J.-C., Jiang, Y., Xie, P., Huang, F., Chen, H., and Zhang, N.

(2024). Knowledge mechanisms in large language models: A survey and perspective.

Ying, J., Cao, Y., Xiong, K., Cui, L., He, Y., and Liu, Y. (2024). Intuitive or dependent? investigating LLMs’ behavior style to conflicting prompts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, pages 4221–4246.

Yu, H., Gan, A., Zhang, K., Tong, S., Liu, Q., and Liu, Z. (2024). Evaluation of retrieval-augmented generation: A survey.

Zhan, P., Xu, Z., Tan, Q., Song, J., and Xie, R. (2024). Unveiling the lexical sensitivity of LLMs: Combinatorial optimization for prompt enhancement. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5128–5154, Miami, Florida, USA. Association for Computational Linguistics.

Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., and Artzi, Y. (2020). BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*.

Zheng, M., Pei, J., Logeswaran, L., Lee, M., and Jurgens, D. (2024). When “a helpful assistant” is not really helpful: Personas in system prompts do not improve performances of large language models. In

Findings of the Association for Computational Linguistics: EMNLP 2024, pages 15126–15154.

Zhou, J., Lu, T., Mishra, S., Brahma, S., Basu, S., Luan, Y., Zhou, D., and Hou, L. (2023). Instruction-following evaluation for large language models.

Zhuo, J., Zhang, S., Fang, X., Duan, H., Lin, D., and Chen, K. (2024). ProSA: Assessing and understanding the prompt sensitivity of LLMs. In *Findings of the Association for Computational Linguistics: EMNLP 2024*.